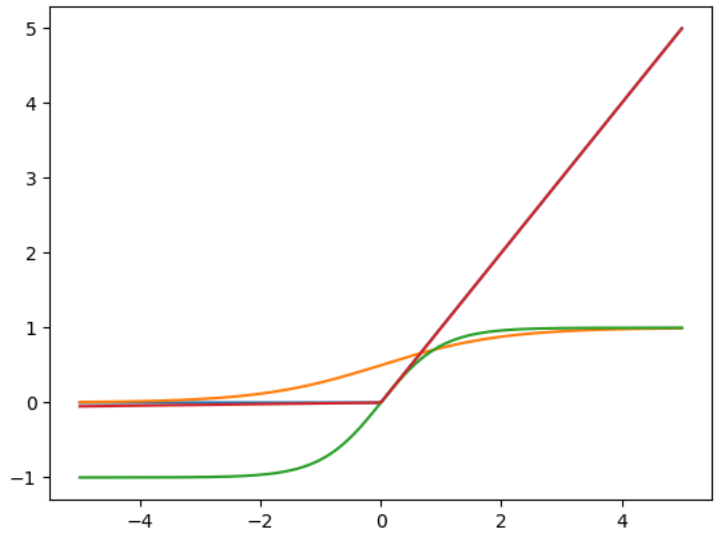
```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
x=np.linspace(-5,5,300)
relu=lambda x:np.maximum(0,x)
sigmoid=lambda x:1/(1+np.exp(-x))
tanh=np.tanh
leaky=lambda x:np.where(x>0,x,0.01*x)
plt.plot(x,relu(x))
plt.plot(x,sigmoid(x))
plt.plot(x,tanh(x))
plt.plot(x,leaky(x))
```

```
[<matplotlib.lines.Line2D at 0x7d4280676120>]
```




```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
(x_train, y_train), (x_test, y_test) = keras.datasets.fashion_mnist.load_data()

x_train = x_train.reshape(-1, 784)
x_test = x_test.reshape(-1, 784)

x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0

model = keras.Sequential([
    keras.Input(shape=(784,)),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])

model.summary()
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
dense_6 (Dense)	(None, 128)	100,480
dropout_4 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8,256
dropout_5 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)
Trainable params: 109,386 (427.29 KB)
Non-trainable params: 0 (0.00 B)

```
import tensorflow as tf
from tensorflow import keras
```

```
from tensorflow.keras import layers
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = x_train.reshape(-1,784)
x_test = x_test.reshape(-1,784)
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
print(f'Train{x_train.shape} Test: {x_test.shape}')
model = keras.Sequential([
    keras.Input(shape=(784,)),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.2),
    layers.Dense(10, activation='softmax')
])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'],
)
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True,verbose=1)
reduce_lr =ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, min_lr=1e-6, verbose=1)
history = model.fit(
    x_train,
    y_train,
    batch_size=128,
    epochs=50,
    validation_split=0.1,
    callbacks=[early_stopping, reduce_lr],
    verbose=1
)
```

```
Train(60000, 784) Test: (10000, 784)
Epoch 1/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.8583 - loss: 0.4734 - val_accuracy: 0.9605 - val_loss: 0.1451 - learn
Epoch 2/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9393 - loss: 0.2091 - val_accuracy: 0.9692 - val_loss: 0.1038 - learn
Epoch 3/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9537 - loss: 0.1558 - val_accuracy: 0.9730 - val_loss: 0.0857 - learn
Epoch 4/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9609 - loss: 0.1327 - val_accuracy: 0.9752 - val_loss: 0.0786 - learn
Epoch 5/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9667 - loss: 0.1108 - val_accuracy: 0.9753 - val_loss: 0.0733 - learn
Epoch 6/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9696 - loss: 0.0978 - val_accuracy: 0.9770 - val_loss: 0.0742 - learn
Epoch 7/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9730 - loss: 0.0882 - val_accuracy: 0.9770 - val_loss: 0.0702 - learn
Epoch 8/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9753 - loss: 0.0809 - val_accuracy: 0.9808 - val_loss: 0.0645 - learn
Epoch 9/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9764 - loss: 0.0743 - val_accuracy: 0.9815 - val_loss: 0.0656 - learn
Epoch 10/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9781 - loss: 0.0674 - val_accuracy: 0.9795 - val_loss: 0.0680 - learn
Epoch 11/50
415/422 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9795 - loss: 0.0637
Epoch 11: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9797 - loss: 0.0635 - val_accuracy: 0.9808 - val_loss: 0.0648 - learn
Epoch 12/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9843 - loss: 0.0493 - val_accuracy: 0.9815 - val_loss: 0.0606 - learn
Epoch 13/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 4s 9ms/step - accuracy: 0.9845 - loss: 0.0476 - val_accuracy: 0.9823 - val_loss: 0.0612 - learn
Epoch 14/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9860 - loss: 0.0439 - val_accuracy: 0.9833 - val_loss: 0.0582 - learn
Epoch 15/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9861 - loss: 0.0437 - val_accuracy: 0.9837 - val_loss: 0.0570 - learn
Epoch 16/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9864 - loss: 0.0421 - val_accuracy: 0.9820 - val_loss: 0.0592 - learn
Epoch 17/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9874 - loss: 0.0390 - val_accuracy: 0.9823 - val_loss: 0.0598 - learn
Epoch 18/50
419/422 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9878 - loss: 0.0387
Epoch 18: ReduceLROnPlateau reducing learning rate to 0.0002500000118743628.
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9879 - loss: 0.0384 - val_accuracy: 0.9833 - val_loss: 0.0581 - learn
Epoch 19/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 5s 6ms/step - accuracy: 0.9896 - loss: 0.0310 - val_accuracy: 0.9825 - val_loss: 0.0600 - learn
Epoch 20/50
422/422 ━━━━━━━━━━━━━━━━━━━━ 3s 6ms/step - accuracy: 0.9900 - loss: 0.0308 - val_accuracy: 0.9827 - val_loss: 0.0593 - learn
Epoch 20: early stopping
Restoring model weights from the end of the best epoch: 15.
```

```
Start coding or generate with AI.
```